Sparse Dynamical Modeling of Baseball Pitching Kinematics from Vision-Derived Pose

Alexander C. Yu, M.S. Student

***Abstract*—This paper develops a reproducible pipeline for converting vision-derived human motion into sparse nonlinear dynamical models, together with an evaluation protocol in which every reported score is paired with the no-model control for its own task. The workflow uses standardized skeletal pose, biomechanical feature extraction, and Sparse Identification of Nonlinear Dynamical Systems, applied to 140 MediaPipe-derived MLB pitching clips from a single broadcast view. The central result separates two tasks that prior evaluations conflated. Free-running reconstruction fails: no configuration outperforms a frozen zero-dynamics predictor, and accuracy degrades monotonically with model complexity. Short-horizon prediction succeeds: a delay-augmented state reaches 0.41 skill over persistence and the project's first positive derivative R2, from 32 sparse terms, clearing an unfitted momentum control. Candidate causes of the failure are excluded one at a time — phase, clip heterogeneity, the pose noise floor, and the sparse estimator — leaving the state representation, which the delay result then confirms. Five intermediate results were false positives that survived only because a control or an aggregation was missing; each is reported with the check that overturned it, and that protocol is offered as a contribution alongside the pipeline. Release velocity and pitch type are not predictable from any of five representations, so no biomechanical claim is made.**

***Index Terms*—sparse identification of nonlinear dynamics, SINDy, human pose estimation, sports biomechanics, model evaluation, baseline controls, time-series forecasting**

I. Introduction

Baseball pitching is a high-speed, multi-segment movement involving coordinated lower-body drive, trunk rotation, shoulder and elbow motion, stride mechanics, and rapid distal segment acceleration. Traditional biomechanical analysis often relies on laboratory motion capture, which produces high-quality trajectories but is expensive and difficult to scale; the canonical kinetic descriptions of pitching mechanics were established this way [1], and the sequence they describe — stride, pelvis rotation, upper-torso rotation, elbow extension, shoulder internal rotation, wrist flexion — is the proximal-to-distal kinetic chain this paper tests against in Section VII [2]. Broadcast or locally segmented video is more accessible, but vision-derived pose introduces noise, camera-view bias, missing landmarks, and two-dimensional projection limits. Markerless pose estimation has been validated against marker-based capture for gait, where joint-angle agreement is good for large sagittal motions and degrades for smaller out-of-plane rotations [3]; broadcast footage is a substantially harder case than the controlled multi-camera setting used in that comparison.

A. C. Yu is with the Department of Electrical and Computer Engineering, Rutgers University, New Brunswick, NJ 08901 USA (e-mail: acy29@scarletmail.rutgers.edu).

This project investigates whether a machine-vision and sparse-dynamics workflow can produce reproducible mathematical artifacts from baseball pitching motion. The goal is not to claim that the current dataset explains elite pitching mechanics. The goal is to establish a defensible technical pipeline that can ingest pose trajectories, condition them into stable feature matrices, estimate derivatives, and fit sparse dynamical models that can be compared across preprocessing and modeling configurations under a common evaluation protocol.

A. Related work

Sparse identification of nonlinear dynamics [4] has been extended in directions that matter here: the weak formulation for noise robustness [5], [6], bootstrap aggregation for the low-data high-noise regime [7], and relaxed-regularization optimizers [8]. All are available in PySINDy [9], [10] and all are evaluated in Section V.

Closest to this work in application is Rosenberg et al. [11], which fits sparse hybrid dynamics to centre-of-mass trajectories during walking and reports individual-specific, interpretable models. Two differences shape what is achievable. That work uses laboratory motion capture, where the measurement noise is far below the signal, and it models a periodic gait cycle, where a low-dimensional template is a good description. Pitching is a single ballistic transient recorded here from broadcast video, and Sections V to 6.8 show both differences bite.

For the pose layer, markerless estimation has been validated against marker-based capture for gait, with good agreement on large sagittal motions and degradation on smaller out-of-plane rotations [3]; broadcast footage is a harder case again. On this dataset specifically, Piergiovanni and Ryoo [12] predict pitch speed and type directly from video and describe the task as extremely difficult, which is the prior against which Section VIII should be read. Laboratory studies do identify kinematic correlates of ball velocity [13] and of trunk-rotation timing [14], so the question this paper can address is not whether such correlates exist but whether they survive this medium.

The delay-coordinate results in Sections VI and VI.9 rest on Takens' embedding theorem [15] and the practice of obtaining linear models of nonlinear dynamics from delay coordinates [16], with dynamic mode decomposition [17] as the spectral counterpart used in Section VIII.

B. Research questions

1. Can raw or vision-derived skeletal trajectories be converted into standardized baseball pitching feature matrices without manual per-clip intervention?
2. Can derivative matrices and sparse coefficient matrices be regenerated reproducibly from those feature matrices?
3. Are the fitted SINDy systems numerically stable on held-out baseball clips?
4. Do any of the fitted systems outperform a no-dynamics control baseline on held-out free-running reconstruction?
5. Do they outperform the corresponding no-model control on short-horizon prediction, and does the answer differ from RQ4?
6. Is the limiting factor the model class, the state representation, the clip heterogeneity, or the pose noise floor?

II. Data

A. Stage A validation data

Stage A uses UTD-MHAD [18] as a proxy human-motion dataset. The selected proxy actions are labeled as `batting_like` and `pitching_like` to test rotational and ballistic motion processing. These labels are methodological categories and do not imply that UTD-MHAD motions reproduce baseball mechanics.

B. Stage B baseball data

Stage B uses local baseball clips drawn from the MLB-YouTube dataset [12], represented in `data/baseball/metadata/clip_manifest.csv`, with local-media split files under `data/baseball/splits/` and standardized pose payloads registered under `data/baseball/standardized_pose/`. MediaPipe pose exports are imported into a 15-joint schema compatible with the Stage A skeleton format.

The current pitching pool is **140 clips** (`data/baseball/splits/pitching_local_pool.csv`), split into **111 training clips** (`pitching_local_train.csv`) and **29 held-out test clips** (`pitching_local_test.csv`). All clips come from a single consistent camera view (`broadcast_centerfield`). The effective sampling interval is `dt = 0.016683 s` (approximately 59.9 fps).

Each clip additionally carries `pitch_type` and `pitch_speed_mph` labels in the pool CSV. These labels are not used by the dynamics pipeline described here; they are recorded for the label-validated representation study scoped in `docs/superpowers/specs/2026-06-23-representation-study-design.md`. Piergiovanni and Ryoo [12] attempt pitch-speed and pitch-type prediction from the same broadcast footage and describe it as extremely difficult, which is the relevant prior for that study.

Generated arrays, processed features, models, and result summaries are intentionally excluded from version control by `.gitignore`. They are reproducible from the commands in Section XII.

III. Pipeline

The workflow has two stages. Stage A validates the general motion-processing and sparse-modeling infrastructure on UTD-MHAD — data loading, skeleton standardization, smoothing, derivative estimation, model fitting, coefficient export — so its artifacts are infrastructure checks rather than baseball findings. Stage B replaces only the input-pose layer with baseball-specific standardized pose and keeps everything downstream, which keeps video and pose-estimation issues separate from the modeling workflow. The pipeline is described here only as far as the results require; Section XII gives the commands that regenerate every number.

A. Pose standardization

Each baseball pose clip is stored as a compressed `.npz` payload containing:

- `coordinates`: pose array with shape `(time, joints, coordinate_dims)`
- `joint_names`: ordered joint labels
- `metadata`: JSON-serializable clip and pose-source metadata

The preferred joint schema contains 15 joints: head, neck, spine, shoulders, elbows, wrists, hips, knees, and ankles. The current adapter maps the larger MediaPipe landmark set into this schema. Pose is produced by the MediaPipe framework [19] using its BlazePose topology [20], which is a monocular on-device estimator rather than a calibrated multi-camera system.

B. Preprocessing

The baseball preprocessing script repairs leading and trailing missing values by edge fill, interpolates internal gaps, centers trajectories at the hip-center proxy, optionally applies orientation and scale normalization, smooths coordinates with a Savitzky-Golay filter [21], and trims low-motion frames around the action window.

The operative baseball configuration is `config/preprocessing.baseball.unaligned.handedness.yaml`, which uses:

- center joint: `hip_center`
- orientation normalization: disabled
- scale normalization: disabled
- internal gap interpolation: enabled, maximum gap 30 frames

- coordinate smoothing: Savitzky-Golay, window length 11, polynomial order 3
- motion trimming: enabled, energy threshold ratio 0.2, 4 padding frames, minimum 25 frames

C. Feature extraction

The current Stage B pitching state vector has 10 variables:

1. `shoulder_rotation_angle`
2. `hip_rotation_angle`
3. `hip_shoulder_separation_angle`
4. `dominant_elbow_flexion_angle`
5. `trail_elbow_flexion_angle`
6. `dominant_knee_flexion_angle`
7. `trail_knee_flexion_angle`
8. `lead_knee_lift`
9. `stride_ankle_separation`
10. `dominant_wrist_speed`

The state is chosen to cover the segments the throwing sequence runs through [2]: stride via `stride_ankle_separation` and `lead_knee_lift`, pelvis and trunk via the two rotation angles and their difference, and the arm via elbow flexion and wrist speed. `hip_shoulder_separation_angle` is included because pelvis-to-trunk separation and the timing of trunk rotation are the parameters laboratory work associates with ball velocity and with shoulder loading [13], [14].

Features are canonicalized by handedness: the configuration sets `default_handedness: R` and `infer_handedness_for_pitching: true`, so dominant/trail designations are resolved per clip rather than fixed to a body side. Angles are unwrapped (`angle_unwrap: true`).

Event alignment is **disabled** in the operative configuration (`features.alignment.enabled: false`). An earlier configuration track aligned pitching clips around stride-ankle separation with lead-knee lift as a secondary signal; that track is retained as `config/preprocessing.baseball.aligned.yaml` but is not the operative default. Section A records why.

Derivatives are estimated with Savitzky-Golay differentiation using window length 9 and polynomial order 3.

D. Sparse model identification

Sparse models are fit by the SINDy method of Brunton et al. [4], which selects a parsimonious set of library terms for `dx/dt` by sequentially thresholded least squares (STLSQ). The implementation is PySINDy [9], [10], using a polynomial library with bias and interaction terms, feature scaling enabled, and STLSQ sparse regression. The operative configuration uses:

- polynomial degree: 1
- STLSQ alpha: 0.001
- maximum STLSQ iterations: 20
- model-selection metric: sparsity-weighted R2 (`selection_metric: sparsity_weighted_r2`)
- complexity penalty: 5e-05 per term
- validation fraction for selection: 0.22 (24 of 111 training clips), seed 7

The STLSQ threshold is **not fixed**; it is swept over `{0.1, 0.15, 0.2, 0.3, 0.5}` and selected by the sparsity-weighted score on the held-out selection split. For the operative first-order configuration the sweep selects **threshold 0.2**, yielding **26 equation terms**. Section IX documents a degeneracy in this selection rule.

A weak-form variant is available at `config/preprocessing.baseball.unaligned.handedness.weakform.yaml`, gated by `modeling.weak_form.enabled`. This uses the weak, integral formulation of Messenger and Bortz [5], [6], which replaces pointwise derivative estimates with integrals against test functions and is therefore substantially more robust to measurement noise — the motivation for trying it on broadcast-derived pose. It fits a `WeakPDELibrary` SINDy over stacked per-trajectory weak systems across all 111 training clips, then transplants the resulting coefficients into a plain polynomial-library SINDy so the standard predict and simulate paths apply unchanged. Because `WeakPDELibrary` places its test-function subdomains via the global NumPy RNG with no seed parameter, a single draw is neither reproducible nor low-variance; the implementation seeds locally (restoring prior RNG state) and stacks `n_seeds` independent subdomain draws. The operative weak-form settings are `K: 100`, `seed: 7`, `n_seeds: 5`.

STLSQ is one sparse optimizer among several; relaxed-regularization alternatives are surveyed and unified by Champion et al. [8], and Section VI reports what changing the optimizer does here. An ensemble variant at `config/preprocessing.baseball.unaligned.handedness.ensemble.yaml` wraps the optimizer in bootstrap aggregation following Ensemble-SINDy [7], which is intended to improve robustness in the low-data, high-noise regime that applies here.

The model outputs include equations, coefficient matrices, model-selection logs, scaler metadata, per-trial metrics, and summary CSVs.

IV. Evaluation protocol

This section is placed before the results because the protocol, not any single number, is what makes them interpretable. Five of this project's intermediate findings were false positives that only a control could catch, and Section IX documents them. Held-out evaluation computes one-step derivative-prediction metrics and free-running reconstruction metrics. Reconstruction integrates the fitted system forward from the first observed state for `reconstruction_frames: 40` frames using the **RK4** integrator (`evaluation.integrator: rk4`; an `euler` option is retained and was marginally worse, core R2 -29.99 vs -29.56 on the first-order baseline).

Two reconstruction metrics are reported:

- **Whole-state reconstruction R2** — variance-weighted across all 10 state columns. This metric is *not* comparable across different state-vector definitions, because high-variance columns dominate it. For the 10-D first-order state, `dominant_wrist_speed` inflates the score.
- **Core-position reconstruction R2 / NRMSE** — a state-comparable metric computed only over the 8 shared `CORE_POSITIONS` columns (all state variables except `hip_shoulder_separation_angle` and `dominant_wrist_speed`), using `r2_score(..., multioutput="uniform_average")` so no single column dominates. NRMSE normalizes each column's RMSE by that column's standard deviation, making it unit-free.

Core-position R2 is additionally reported at rollout horizons `{5, 10, 20, 40}` frames from the same forward simulation.

Control baseline. Reconstruction R2 is measured against the per-window mean, which is a harsh reference when the core positions move little over a short rollout. To make the reconstruction numbers interpretable, `src/preprocessing/evaluate_static_baseline.py` scores a **frozen model** — zero derivative, so the state is held at its first observed value for the whole rollout — under the identical metric, horizons, and test split. This encodes no dynamics whatsoever and therefore establishes the floor that any fitted model must clear for its reconstruction score to carry information.

Simulation stability (finite and bounded trajectories) is reported separately. Stability is a necessary but very weak condition and is not evidence of predictive accuracy: a model whose coefficients are near zero is bounded by construction.

Short-horizon prediction and skill over persistence. Free-running reconstruction and one-step prediction are different tasks with non-comparable scores, and each needs its own control. For one-step prediction the correct control is not the frozen model but **persistence** — predict `x(t) = x(t-1)`. At approximately 60 fps consecutive frames are nearly identical, so persistence is a very strong baseline: it alone scores R2 = 0.968 on the state. State R2 is therefore saturated by the near-identity component and cannot separate models. The reported metric for this task is instead **skill over persistence** on the increment:

```
skill = 1 - SSE(model) / SSE(zero-increment)
```

where the target is `delta(t) = x(t+1) - x(t)`. Expressing accuracy against a naive benchmark rather than in absolute units is standard practice in forecast evaluation [22]. This is the standard mean-square-error skill score, in which accuracy is expressed relative to a trivial reference forecast rather than in absolute terms [23]; persistence is the reference here, and the climatological analogue — the window mean — is what the reconstruction R2 above uses.

TABLE I
Training summary for the operative first-order configuration, 111 training clips.

quantity	value
training clips	111
feature count	10
selected STLSQ threshold	0.2
selected equation terms	26
global derivative R2 (train)	0.0063
mean trial derivative R2 (train)	-0.0136
mean trial derivative RMSE (train)	10.9599

Skill is 0 for persistence by construction, and positive skill means predictive structure beyond "nothing changes."

`evaluate_baseball_sindy.py` computes this on every run, as `persistence_skill_pooled` and `persistence_skill_uniform` per clip and `mean_persistence_skill_*` in the summary, so the metric the project judges models by is emitted by the pipeline itself rather than by a separate diagnostic. The reconstruction metrics remain alongside it.

Aggregation is reported two ways, and disagreement is treated as a null result. Core positions span roughly a 13x range in per-frame signal scale (`lead_knee_lift` 0.012 versus `dominant_elbow_flexion_angle` 0.158, about 160x in variance), so pooled sum-of-squares is decided almost entirely by the largest-amplitude columns while uniform per-column averaging weights all eight equally. Every skill number in Section V is reported under both. Where the two disagree in sign, this paper reports no win rather than selecting the favorable aggregation. `core_position_metrics` uses uniform averaging, so uniform is the figure comparable to the rest of the project.

V. Free-running reconstruction fails

All results below were produced on 2026-08-12 against the same 140-clip pool (111 train / 29 test), RK4 integrator, and 40-frame reconstruction. Every number is backed by a CSV under `results/`; the consolidated table is `results/evaluation/baseball/model_comparison_2026-08-12.csv`.

A. The operative configuration

Training summary (`results/equations/baseball/train/summary.csv`, snapshot `train_plain_first_order_2026-08-12/`):

Held-out summary (`results/evaluation/baseball/train_on_test/summary.csv`). The skill rows make the operative model's position explicit: it has no short-horizon predictive structure beyond persistence, which is what Section VI attributes to the state rather than the estimator, and Section VII recovers by adding history.

B. Every model against the frozen control

`core_position_recon_r2` is the state-comparable metric; higher (less negative) is better.

TABLE II
HELD-OUT EVALUATION OF THE OPERATIVE FIRST-ORDER CONFIGURATION, 29 TEST CLIPS.

quantity	value
held-out clips	29
mean derivative R2	-0.0184
mean derivative RMSE	9.8448
skill over persistence, pooled	+0.0014
skill over persistence, uniform	-0.0041
stable simulation fraction	1.0
whole-state reconstruction R2	-2.7981
whole-state reconstruction RMSE	0.7232
core-position reconstruction R2	-29.5553
core-position reconstruction NRMSE	3.2544

Those are single-split figures. Because every configuration was scored on the same 29 held-out clips, the comparison against the control is naturally **paired**, and a paired statistic is far tighter than resampling two means independently. Bootstrapping [24] the per-clip difference over clips — not over frames, since frames within a clip are strongly dependent and would understate the uncertainty — gives (`src/analysis/paired_control_intervals.py`):

Both intervals exclude zero, so the headline claim survives: these models are worse than doing nothing, not merely indistinguishable from it.

One nuance the means hide. Weak-form actually beats the control on **15 of 29 clips**, a bare majority, yet its average is worse — its losses are far larger than its wins. The aggregate conclusion is therefore driven by tail behaviour on a minority of clips rather than by uniform inferiority, which is worth knowing for anyone tempted to read the mean as "always worse".

Short-horizon core R2, plain vs weak-form vs control:

Three observations follow directly:

1. **No fitted model beats the frozen control at any horizon.** The best fitted model (weak-form, core R2 -7.89) is outscored by a predictor that holds the initial pose constant (-5.57). Under this metric, every dynamical model tried is worse than assuming the pitcher does not move.
2. **The ordering is monotonic in model complexity, in the wrong direction.** Core R2 degrades as term count rises ($0 \to 1 \to 26 \to 40 \to 101 \to 131$ terms). Added expressivity strictly hurts held-out reconstruction, which is the signature of fitting noise rather than dynamics.
3. **Stability is uninformative here.** Every configuration reports a stable simulation fraction of 1.0, including models that are numerically bounded only because their coefficients are near zero.

C. Rollout horizon

The same increment model rolled out autoregressively (`results/analysis/increment_skill/part_b_horizon_crossover.csv`):

A separate verification confirms the rollout failure is genuine rather than an implementation artifact. Teacher-forcing the same model, so true history is supplied at each step and no error compounds, scores +0.7569 core R2 against persistence's +0.8453 (`results/evaluation/baseball/forecast_baselines/summary.csv`). The map is sound; free-running divergence is real error compounding — the quadratic-in-horizon growth that arises whenever a model is fed its own predictions instead of ground truth, and the reason one-step accuracy does not imply rollout accuracy [25]. Note that this teacher-forced score is itself below persistence, so it is not an additional positive result — it is only evidence that `rollout_delay` is correct.

For completeness, the free-running comparison against all controls under a common seed and scoring window:

The `mean_pitch` result is worth noting on its own: a template asserting that every pitch follows the average training trajectory scores -45.01, so at this pose quality pitching is not stereotyped enough for template matching either.

VI. The cause is the state, not the model

Four candidate explanations for the failure are excluded in turn, each by direct test.

A. Phase and clip heterogeneity

Four candidate explanations for the near-zero derivative fit were tested directly, using plain ridge regression rather than STLSQ so that sparse-selection artifacts cannot confound the comparison, with GroupKFold over clips so no clip appears in both train and test, since frames inside a clip are strongly dependent and a naive fold split over a time series is optimistic [26] (`src/preprocessing/diagnose_derivative_fit.py`, `results/analysis/derivative_fit_diagnostics.csv`):

Two conclusions follow.

The non-autonomous hypothesis is refuted. Adding normalized phase φ — as an additive term, as a modulating interaction $A(\varphi)x$, and as piecewise-in-phase bins — produces no gain in any form. The failure of `dx/dt = f(x)` is not explained by the map being multivalued in phase.

The state representation, not clip heterogeneity, is the binding constraint. The per-clip oracle scores only +0.25 *in sample*, lower than a delay-embedded model scores on *held-out* clips (+0.69). A single clip cannot be fit well in the plain 10-D state even when the model is allowed to see it. This rules out the per-pitcher, per-pitch-type, and per-phase conditioning directions, which all address heterogeneity.

The delay-embedded rows follow Takens [15], whose embedding theorem motivates reconstructing a state from lagged observations of a single trajectory, and the specific use of delay coordinates to obtain a linear model of

TABLE III
Held-out comparison of every fitted configuration against the frozen zero-dynamics control. Core R2 degrades monotonically as term count rises.

model	terms	core R2	core NRMSE	whole-state R2
frozen control (no dynamics)	**0**	**-5.5707**	**2.2158**	n/a
first-order, weak-form, multi-seed	1	-7.8922	2.3781	-6.7185
first-order, plain (operative)	26	-29.5553	3.2544	-2.7981
first-order, ensemble	40	-36.9650	3.5159	-3.0544
second-order, plain	101	-2269.04	19.1044	-6.0350
second-order, ensemble	131	-2428.87	19.2161	-6.2521

TABLE IV
Paired bootstrap over clips, 10,000 resamples. Both intervals exclude zero, so both models are worse than the frozen control by more than sampling noise.

model	mean difference from control	95% CI	clips beating control
first-order, plain (operative)	-23.98	[-48.21, -4.22]	11/29
first-order, weak-form, multi-seed	-2.32	[-4.98, -0.27]	15/29

TABLE V
Core-position reconstruction R2 by rollout horizon. The frozen control outscores both fitted models at every horizon.

horizon (frames)	plain first-order	weak-form	frozen control
5	-75.47	-2.6818	**-2.1619**
10	-47.81	-5.4027	**-2.8330**
20	-37.99	-4.9043	**-4.2264**
40	-29.56	-7.8922	**-5.5707**

TABLE VI
Rolled-out increment model against persistence by horizon. Pooled and uniform skill disagree in sign at every horizon, so no multi-step win is claimed.

H (frames)	H (sec)	clips	skill (pooled)	skill (uniform)	core R2 model	core R2 persistence
5	0.083	29	+0.1751	-68.15	-354.00	-8.38
10	0.167	29	+0.1906	-23.47	-110.86	-6.76
20	0.334	29	+0.2145	-24.82	-68.56	-5.92
40	0.667	29	+0.1269	-17.19	-43.36	-5.63
80	1.335	28	+0.1385	-7.72	-33.60	-5.14
160	2.669	21	+0.1517	-4.13	-9.30	-3.16
240	4.004	13	+0.1438	-0.4955	-1.87	-2.73

TABLE VII
Free-running forecast comparison under a common seed and scoring window. The frozen control is the best available method.

method	core R2	reading
frozen (no dynamics)	**-5.56**	best free-running method
`delay_linear` rolled out	-42.08	the +0.69 derivative representation
`mean_pitch` template	-45.01	phase-warped mean of the training clips
`constant_velocity`	-1096.28	extrapolation from the seed velocity

nonlinear dynamics follows the HAVOK construction of Brunton et al. [16]. Their apparent gain must nonetheless be discounted: an unfitted backward difference already reaches +0.3507, so roughly half of the +0.69 is numerical differentiation recovered from position history rather than learned dynamics.

B. The pose noise floor

Per-feature signal and predictability across all 140 clips (`results/analysis/increment_skill/part_c_noise_floor.csv`):

Increment lag-1 autocorrelation ranges from 0.35 to 0.88, so increments are far from white and carry genuinely predictable structure; signal-to-noise is 2.2 to 4.4 throughout. **The noise floor does not explain the failures in**

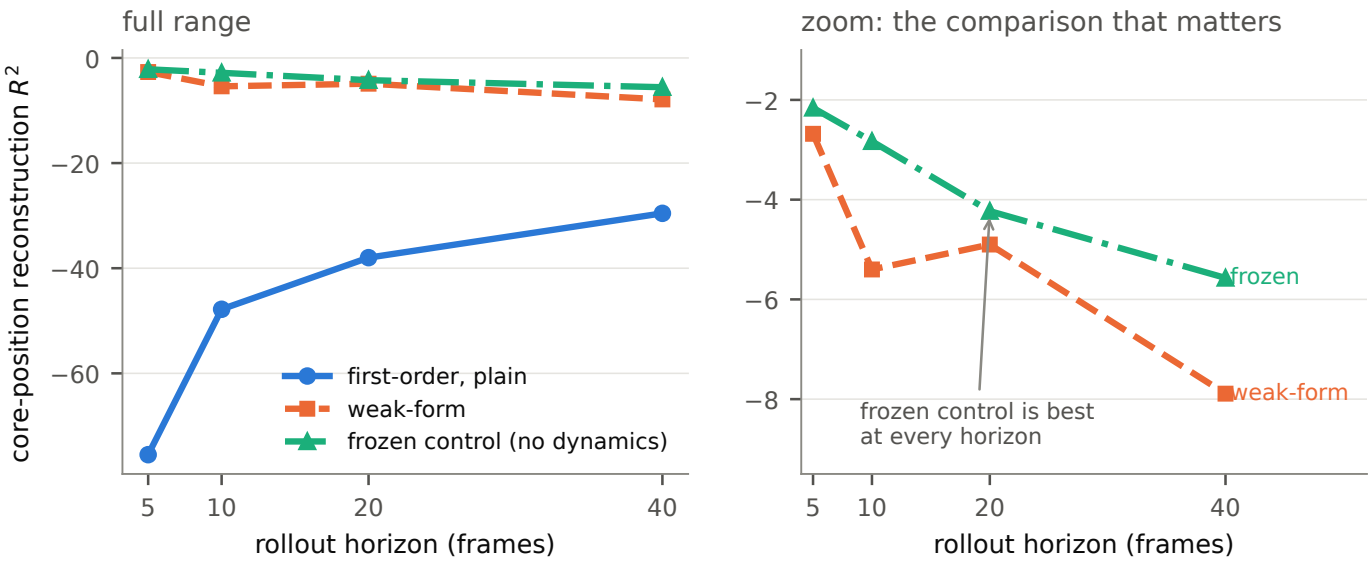


Fig. 1. Core-position reconstruction R2 by rollout horizon. The left panel shows the full range; the right panel zooms to the comparison that matters. The frozen control outscores both fitted models at every horizon.

TABLE VIII
HELD-OUT DERIVATIVE R2 BY STATE REPRESENTATION, PLAIN RIDGE WITH GROUPKFOLD OVER CLIPS. PHASE ADDS NOTHING; HALF OF THE DELAY-EMBEDDING GAIN IS RECOVERED NUMERICAL DIFFERENTIATION.

representation	held-out derivative R2	reading
`global_linear` — [1, x]	-0.0004	structural match to the operative degree-1 SINDy
`global_linear_phase` — [1, x, φ]	-0.0003	no effect
`global_phase_interact` — A(φ)x	+0.0071	no meaningful gain
`phase_binned_k4` / `k8`	+0.0019 / -0.0008	no gain
`delay_embed` τ=1, d=3	**+0.6901**	large apparent gain
`finite_diff_control` (unfitted backward difference)	+0.3507	**half the gain is rediscovered differentiation**
`per_clip_within` (fit early 70%, predict late 30%)	-38.89	within-clip temporal extrapolation fails
`per_clip_oracle` (in-sample)	+0.2500	upper bound for a per-clip model

TABLE IX
PER-FEATURE SIGNAL SCALE, HIGH-FREQUENCY RESIDUAL, AND INCREMENT AUTOCORRELATION ACROSS ALL 140 CLIPS. THE NOISE FLOOR DOES NOT BOUND SHORT-HORIZON PREDICTION.

feature	core	signal/frame	HF residual	SNR	$ACF_1(\Delta)$
`lead_knee_lift`	yes	0.0124	0.0028	**4.44**	**0.8816**
`stride_ankle_separation`	yes	0.0167	0.0050	3.35	0.7316
`hip_rotation_angle`	yes	0.1131	0.0355	3.19	0.7716
`shoulder_rotation_angle`	yes	0.1145	0.0386	2.97	0.7429
`trail_knee_flexion_angle`	yes	0.1184	0.0429	2.76	0.6247
`dominant_knee_flexion_angle`	yes	0.1270	0.0488	2.60	0.5795
`trail_elbow_flexion_angle`	yes	0.1529	0.0665	2.30	0.4523
`hip_shoulder_separation_angle`	no	0.0758	0.0330	2.30	0.5395
`dominant_elbow_flexion_angle`	yes	0.1576	0.0690	2.28	0.4303
`dominant_wrist_speed`	no	0.7066	0.3184	2.22	0.3509

Sections V and VI.6, and short-horizon prediction is not capped by pose quality at this level. Higher-quality pose remains desirable for biomechanical interpretation, but it is not a precondition for the Section VII result.

The residual estimate is an upper bound on achievable precision, not a measurement of raw MediaPipe jitter: the features are already Savitzky-Golay smoothed, so this measures leftover high-frequency content in the modeled

features. It is a floor on the floor.

C. The sparse estimator

Section VII established short-horizon structure using a delay-embedded ridge model, which is not what this pipeline produces. The question that determines how broadly the result can be claimed is whether the fitted STLSQ equations capture that structure. `src/preprocessing/evaluate_sindy_increment_skill.py` scores the fitted models on the identical task, with a `savgol_derivative_reference` row giving the ceiling any model fit to those derivative targets could reach.

Uniform skill over persistence, 29 held-out clips:

The answer is no. The operative first-order model has no short-horizon skill whatever, despite the derivative targets it is fit against containing 0.70 of extractable skill. It discards all of it.

The second-order position-increment score of 0.8905 is not a result. Velocity is an input column of the 16-D state, and the identity gate confirms each position derivative loads on its paired rate column, so `d(position)/dt` is read off the input rather than learned. The model ties its own reference to within 0.0003 — the signature of an input-copying identity, not a fitted dynamic. This was anticipated before the run and the control was built in, which is why it was caught immediately rather than reported.

On the non-trivial task for that state — predicting rate increments, i.e. acceleration — the second-order model captures 12 percent of its ceiling. Positive, but small, and it is the same configuration that fails catastrophically on free-running reconstruction (core R2 -2269, Section V).

Correction to the ceiling figure. An earlier version of this section treated the `savgol_derivative_reference` row as the ceiling any model fit to these targets could reach, and concluded that the sparse fit was discarding 0.69 of available skill. That was wrong. `dt * X_dot` is not an achievable ceiling: `X_dot` is estimated by Savitzky-Golay differentiation over a window that includes future frames, so scoring it against the forward increment measures the skill of already knowing the answer, not the skill a model could obtain by predicting it from the state. The reference row is an oracle, not a bound.

The achievable bound is what a dense, minimally regularized linear model of the same state extracts. `src/analysis/diagnose_sparse_fit_loss.py` measures it by holding the design matrix and the target fixed and varying only the estimator (`results/analysis/sparse_fit_loss.csv`):

Sparsification is not the binding loss. Dense ridge and the best STLSQ differ by 0.0002 on the first-order state and 0.0000 on the second-order state, and their confidence intervals overlap almost completely. There is a real but small threshold cost — at the operative threshold 0.2 the first-order fit is zeroed entirely (+0.0005 against +0.0128 at threshold 0.01), which is why the fitted first-order model scored zero — but the amount at stake is 0.013, not 0.69.

The binding constraint is the state's information content. For the first-order state a dense linear model of position alone reaches only +0.0099 [+0.0086, +0.0112]: position does not determine the next increment, so there was almost nothing for the sparse fit to lose. This agrees with Section VI, where plain ridge on `[1, x]` scored — 0.0004 on the derivative.

What does predict the increment is recent history. On the identical target and the identical 29 test clips:

Two consequences for what this paper may claim. First, **the Section VII positive result does not belong to the SINDy pipeline**; it belongs to the data and to a delay-embedded ridge baseline. Second, the loss is localized — but to the *state representation*, not to the sparse fit. `dx/dt = f(x)` over joint positions cannot express what predicts the next increment, and no choice of sparse estimator, threshold, or selection rule changes that.

Skill figures are comparable *within* a row's own reference, not across the first-order and second-order rows: the two configurations compute positions and derivatives with different Savitzky-Golay paths, so their targets and therefore their ceilings differ (0.7032 versus 0.8908 on the same eight core columns).

VII. Short-horizon prediction succeeds

One-step held-out prediction on the 29 test clips (`src/preprocessing/diagnose_increment_skill.py`, `results/analysis/increment_skill/part_a_parameterization.csv`):

This is the project's only positive result that survives its control, and it is positive under both aggregations. The model captures approximately 39 percent of held-out increment variance beyond persistence.

The `R2 (state)` column shows why every earlier evaluation missed it: 0.9777 against persistence's 0.9680. A 39 percent improvement in the predicted dynamics appears as a third-decimal difference once it is measured on the state, because the state metric is dominated by the near-identity component. This is a metric-selection failure, not a modeling one.

The increment parameterization itself contributed nothing (0.2815 versus 0.2816). `RidgeCV` selects alpha of order 1e-4, so shrinkage is negligible, and the two targets span the same linear space. The argument that predicting the next state penalizes the near-identity map is correct in principle but empirically void at this regularization strength. Recording this because the reparameterization was predicted in advance to be the source of the improvement, and it was not — the improvement came entirely from changing the metric.

TABLE X
FITTED SINDY EQUATIONS ON THE SHORT-HORIZON TASK, AGAINST THE CEILING SET BY THEIR OWN DERIVATIVE TARGETS.

model	task	skill	ceiling	fraction of ceiling
first-order SINDy (operative)	position increments	**-0.0010**	0.7032	**0%**
second-order SINDy	position increments	0.8905	0.8908	~100% (see below)
second-order SINDy	rate increments (acceleration)	**0.0826**	0.6911	**12%**
delay ridge (not SINDy)	position increments	0.3874	—	—

TABLE XI
ACHIEVABLE SKILL BY ESTIMATOR OVER 20 RANDOM CLIP-LEVEL RESPLITS, DESIGN MATRIX AND TARGET HELD FIXED, WITH 95% CONFIDENCE INTERVALS. THE DENSE AND SPARSE INTERVALS OVERLAP ALMOST COMPLETELY ON BOTH STATES, SO SPARSIFICATION COSTS NOTHING DETECTABLE.

state	target	RidgeCV (dense)	best STLSQ	STLSQ at operative 0.2
first-order	position increment from position alone	**+0.0099** [+0.0086, +0.0112]	+0.0097 [+0.0083, +0.0111]	+0.0001
second-order	rate increment from position and velocity	**+0.1311** [+0.1181, +0.1440]	+0.1310 [+0.1181, +0.1440]	+0.1246

TABLE XII
THE SAME TARGET AND THE SAME CLIPS, DIFFERING ONLY IN WHETHER THE MODEL IS GIVEN RECENT HISTORY. AN AUTONOMOUS FIRST-ORDER SINDY MODEL HAS NO MECHANISM TO USE IT.

state given to the model	uniform skill over persistence
position only, `[1, x(t)]`	+0.0099
position plus two frames of history	**+0.3874**

TABLE XIII
ONE-STEP HELD-OUT INCREMENT PREDICTION. SKILL OVER PERSISTENCE IS POSITIVE UNDER BOTH AGGREGATIONS, WHILE STATE R2 CANNOT SEPARATE THE MODELS.

model	target	skill (pooled)	skill (uniform)	R2 (state)
persistence (no model)	—	0.0000	0.0000	0.9680
delay ridge	next state x(t+1)	0.2815	**0.3871**	0.9777
delay ridge	increment x(t+1) - x(t)	0.2816	**0.3874**	0.9777

A. Sparse identification on a delay-augmented state

Section VI localizes the failure to the state rather than the estimator. This section tests the implied fix: fit the project's own sparse estimator, STLSQ, on a delay-augmented state and score it against persistence. Design is `[x(t), delta(t-1), ..., delta(t-d+1)]`, target is the next-frame increment of the eight core positions, and history depth is swept rather than assumed (`src/analysis/fit_delay_sindy.py`, `results/analysis/delay_sindy/`).

Sparse identification retains essentially all of the available skill. At depth 3 a 30-term sparse model scores +0.3881 uniform against dense ridge's +0.3877 with 240 coefficients — the sparse fit is not worse, and it is an order of magnitude smaller. This is the first result in this project that belongs to the pipeline's own method rather than to a baseline.

It also beats the unfitted momentum control, which is the bar that matters here. Every discovered equation is dominated by the target's own previous increment, so the relevant control is not persistence but momentum — a joint continuing to move as it was moving, which requires

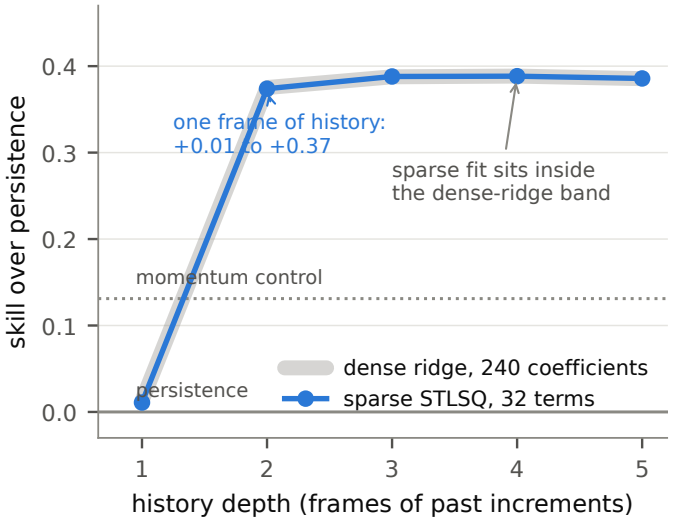


Fig. 2. Skill over persistence against history depth. The dense-ridge band and the sparse fit coincide: 32 terms match 240 coefficients. Almost the entire gain arrives with the first frame of history.

TABLE XIV
SPARSE SKILL OVER PERSISTENCE BY HISTORY DEPTH. ALMOST ALL OF THE GAIN ARRIVES WITH THE FIRST FRAME OF HISTORY, AND THE CURVE SATURATES BY DEPTH 3.

history depth	design dims	STLSQ terms at thr 0.05	skill (pooled)	skill (uniform)
1 (no history)	10	8	+0.0155	+0.0109
2	20	21	+0.2685	+0.3739
3	30	30	**+0.2817**	**+0.3881**
4	40	33	+0.2808	+0.3884
5	50	38	+0.2781	+0.3858

no fitting at all:

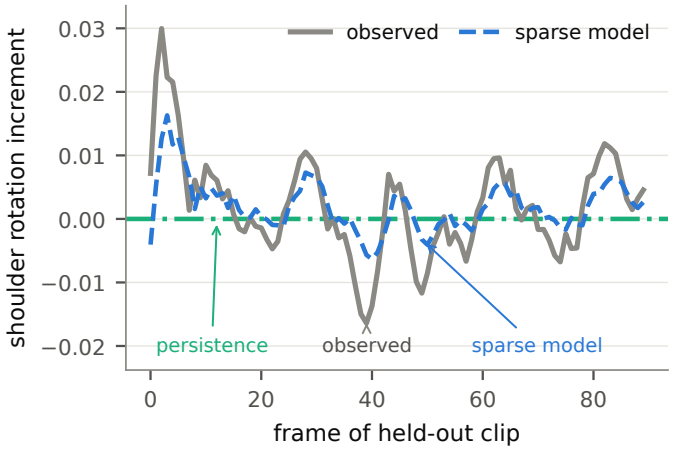


Fig. 3. Observed against predicted one-step increments for one held-out clip, shoulder rotation. Plotted on the increment rather than the state because persistence alone scores 0.968 on the state, so state-space curves overlay and show nothing.

The fitted model clears the momentum control by 0.257 uniform, and both aggregations agree in sign for the first time in this project's forecasting results. Constant-acceleration extrapolation is far worse than either, which is consistent with the noise level in the increments.

B. The pipeline reproduces it

The pipeline itself now produces this result. The findings above came from an analysis script; the pipeline's own default still fitted `dx/dt = f(x)` over positions. The feature extractor now takes a `features.delay_embedding` block (`enabled`, `depth`, `drop_features`), so a delay-augmented state is a first-class configuration rather than something only the analysis code can build. `config/preprocessing.baseball.delay.yaml` is that configuration: depth 3, `hip_shoulder_separation_angle` dropped per the stability finding, and selection on `persistence_skill_uniform`.

Running the standard fit and evaluate commands on it, held out on the same 29 test clips:

Three things change at once. Derivative R2 goes positive for the first time in this project. Skill over persistence reaches +0.4071, slightly above the +0.3881 the analysis script obtained, and the difference is worth noting: the pipeline fits Savitzky-Golay derivatives rather than forward increments directly, so the two routes are not identical and neither is strictly better. And the candidate sweep is **no longer flagged degenerate** — thresholds now separate on skill (+0.3076 down to +0.3026 on the validation clips), so the selection means something for the first time.

The reconstruction metrics on this configuration read -260.7 and must be ignored. A delay state cannot be integrated forward: its delay columns are lagged copies rather than independent degrees of freedom, so a free-running rollout is ill-posed. The evaluator now prints a note to that effect whenever it detects delay columns, so the number cannot quietly be read as a result.

This configuration is offered as the recommended direction, not as a drop-in replacement for the operative one. It answers a different question — one-step prediction rather than trajectory reconstruction — and Section V is the reason that is the right question to be asking.

VIII. WHAT THE DATA DOES NOT SUPPORT

A. Cross-joint coupling and the kinetic chain

Cross-joint coupling contributes, but weakly. Restricting each equation to terms built from its own variable costs 0.0168 uniform and 0.0172 pooled (+0.3713 against +0.3881). The coupling is therefore real and consistent across both aggregations, but it accounts for roughly four percent of the model's skill. The discovered structure is suggestive:

```
delta(shoulder_rotation_angle) = +0.373*d1(shoulder_rotation_angle)
↪  +0.185*d1(hip_rotation_angle)
delta(hip_rotation_angle)      = +0.508*d1(hip_rotation_angle)
↪  +0.092*d1(shoulder_rotation_angle)
```

The hip-to-shoulder coefficient is roughly twice the shoulder-to-hip coefficient, an asymmetry in the direction the proximal-to-distal kinetic chain would predict [1], and `stride_ankle_separation` picks up a `lead_knee_lift` term.

The coupling gain is small but statistically detectable, and that is still not a biomechanical finding. `src/analysis/test_coupling_significance.py` tests it two ways (`results/analysis/coupling_significance/summary.json`). The statistic is the gain itself: uniform skill with all terms minus uniform skill with each equation restricted to its own variable's history.

The null circularly rolls, within each clip, every design column that is not part of the target variable, then refits and recomputes the gain. The roll is applied *per target*: any cross-joint column is some other target's self column,

TABLE XV
FITTED SPARSE MODELS AGAINST UNFITTED MOMENTUM CONTROLS, DEPTH 3, THRESHOLD 0.05. THE FIT ADDS 0.257 UNIFORM SKILL OVER THE STRONGEST UNFITTED CONTROL, SO THE RESULT IS NOT MERELY MOMENTUM.

model	terms	skill (pooled)	skill (uniform)
persistence, zero increment	0	0.0000	0.0000
momentum, `delta_hat(t) = delta(t-1)`	0	-0.0643	+0.1311
momentum, constant acceleration	0	-1.5219	-0.9712
sparse STLSQ, self terms only	20	+0.2645	+0.3713
sparse STLSQ, all terms	30	**+0.2817**	**+0.3881**

TABLE XVI
THE SAME PIPELINE COMMANDS ON THE OPERATIVE AND DELAY-AUGMENTED CONFIGURATIONS. THE DELAY STATE PRODUCES THE PROJECT'S FIRST POSITIVE DERIVATIVE R2 AND A NON-DEGENERATE CANDIDATE SWEEP.

metric	operative first-order	delay-augmented
mean derivative R2	-0.0184	**+0.4411**
skill over persistence, uniform	-0.0041	**+0.4071**
skill over persistence, pooled	+0.0014	+0.2193
selection flagged degenerate	yes	**no**

so a single shared mask would leave nearly all of them untouched. A circular roll also preserves each column's marginal distribution and autocorrelation, where a plain row shuffle would destroy the smoothness that makes these columns predictive at all and so give an unrealistically easy null.

These results use the stabilized 27-column design. On the original design the test was contaminated: cancelling coefficients of order 1e3 on the rank-deficient columns swamped the comparison, diverging splits had to be excluded by hand, and the gain reversed sign across splits. With the derived feature removed, **no splits are excluded** and the picture is consistent.

Across 30 random clip-level resplits, none excluded, the gain averages **+0.0071** with a 95% confidence interval of [+0.0055, +0.0087] and is positive on 29 of 30. Against null distributions with standard deviations of 0.0001 to 0.0003, the four positive splits sit many standard deviations above chance. The cross-joint terms therefore do carry information.

What that does and does not license. The effect is about **1.8 percent** of the model's 0.389 total skill, and one split in five shows none of it, so almost all of what this model does remains each joint's own momentum. More importantly, detecting that shoulder motion is predicted in part by hip motion is not a discovery: the hips and shoulders are mechanically linked through the trunk, and any whole-body coordination would produce it. The proximal-to-distal kinetic chain is a claim about *direction and precedence*, so it was tested directly.

The directional asymmetry does not survive. `src/analysis/test_kinetic_chain_asymmetry.py` orders the state variables from the ground up — `stride_ankle_separation` < `lead_knee_lift` < knee flexion < `hip_rotation_angle` < `shoulder_rotation_angle` < elbow flexion < `dominant_wrist_speed` — and for each equation splits the cross-joint columns into those from more proximal variables and those from more distal ones. Segments at the same level of the chain count for neither direction. Three models are fitted per split, and the statistic is

```
asym = [skill(self + proximal-to-distal) - skill(self)]
     - [skill(self + distal-to-proximal) - skill(self)]
```

The two directions offer very different numbers of columns to different targets — 15, 12, 18, 18, 6, 6, 3, 0 proximal sources against 9, 12, 3, 3, 15, 15, 21, 24 distal ones — so a raw comparison would largely measure that imbalance. The shift null preserves the counts and centres the asymmetry on zero regardless.

Across 30 resplits: proximal-to-distal gains +0.0033, distal-to-proximal gains +0.0022, and the asymmetry is **+0.0011 with a 95% confidence interval of [-0.0004, +0.0026], positive on 18 of 30 splits**. Eighteen of thirty is close to a coin flip, and the interval includes zero.

The two tests disagree, and the disagreement is itself worth recording. The null distributions are extremely tight (standard deviations of 1e-4), so any small positive observation clears them; the null asks only whether the gain differs from shifted-column chance on *that* split. The resplit interval asks whether the effect is consistently positive across different held-out data, which is the question that matters for generalization, and it answers no. Where a tight null and a split-to-split interval disagree, this paper follows the interval.

Conclusion on the kinetic chain: not evidenced. The asymmetry is roughly 0.3 percent of the model's skill and its sign varies across splits. The fitted hip-to-shoulder coefficient being about twice shoulder-to-hip is a property of one coefficient pair in one fit, not a directional effect recoverable from held-out prediction. This paper therefore makes no biomechanical claim about proximal-to-distal sequencing.

B. Release velocity and pitch type

Every result above judges the pipeline by self-prediction of the motion. The clips also carry labels external to the pose — `pitch_speed_mph` (82.1 to 102.7, mean 91.3) and `pitch_type` — which the dynamics pipeline never uses. This section asks whether the representations encode structure that predicts them. A positive answer would be the first

TABLE XVII
PER-TARGET CIRCULAR-SHIFT NULL, 100 DRAWS ON EACH OF FIVE INDEPENDENT HELD-OUT SPLITS, STABILIZED DESIGN. THE GAIN CLEARS THE NULL ON FOUR SPLITS AND IS ABSENT ON THE FIFTH.

held-out split	observed gain	null mean	null sd	p
1	+0.0081	+0.0000	0.0001	0.010
2	+0.0159	-0.0000	0.0002	0.010
3	-0.0011	+0.0000	0.0001	1.000
4	+0.0024	-0.0002	0.0003	0.010
5	+0.0071	+0.0000	0.0002	0.010

TABLE XVIII
SHIFT NULL ON THE DIRECTIONAL ASYMMETRY, 60 DRAWS ON EACH OF FIVE SPLITS. FOUR SPLITS CLEAR THE NULL, BUT THE RESPLIT INTERVAL INCLUDES ZERO AND ONLY 18 OF 30 SPLITS ARE POSITIVE.

held-out split	observed asymmetry	null mean	null sd	p
1	+0.0040	+0.0000	0.0001	0.016
2	+0.0026	+0.0000	0.0003	0.016
3	+0.0128	+0.0000	0.0002	0.016
4	+0.0016	+0.0000	0.0002	0.016
5	-0.0020	+0.0001	0.0002	1.000

validation of this pipeline against a quantity outside its own trajectories.

The protocol was fixed before any result was inspected:

- **Primary endpoint** T1, `pitch_speed_mph` regression, RidgeCV, cross-validated R2.
- **Reference** R0, a 70-dimensional kinematic summary (per-feature mean, spread, extremes, peak timing, mean per-frame speed) containing no dynamical model. Any dynamical representation must beat it to be interesting.
- **Secondary** T2, binary fast (fastball, sinker; 44 clips) versus offspeed (changeup, curveball, knucklecurve, slider; 96 clips), balanced accuracy.
- **Significance** permutation null on the labels, following the procedure of Ojala and Garriga [27]. A result counts only if it beats both the trivial baseline and the null.

All 140 clips, 5-fold stratified cross-validation repeated 20 times, 200 permutations, PCA at 0.95 variance applied identically to every representation (`src/analysis/run_representation_study.py`, `results/analysis/representation_study/`).

The primary endpoint is null for all five representations. None predicts release velocity: mean absolute error spans 5.32 to 5.37 mph against a 5.32 mph mean-predictor baseline, and every p-value lies between 0.33 and 0.64. R0 fails as well, so this is not a case of dynamics failing to add value over a kinematic summary — there is no velocity signal in any of them.

The five span genuinely different descriptions of a clip: descriptive statistics with no dynamics (R0), sparse library coefficients (R1), the same fitted through the noise-robust weak formulation (R2), the spectrum of the best-fit linear operator, that is dynamic mode decomposition [17] (R3), and that spectrum on a delay-embedded state (R4). R4 is included because Section VI shows history is what these trajectories need, so if any spectral description were to carry velocity information it should be that one. It does not (R2 = -0.0201, p = 0.64). R2 is the closest to its baseline at -0.0035, which is exactly the mean-predictor value: its regression is returning the training mean, not finding structure.

The secondary endpoint illustrates why the null matters. R1's balanced accuracy of 0.5504 has a 95% confidence interval that excludes the 0.4919 baseline, which reads as a win. Its permutation p-value is 0.19. The permutation test is the correct arbiter: cross-validation folds share data, so the interval across repeats understates uncertainty — the same mechanism by which a cross-validated score becomes optimistically biased once the same folds have informed a modelling choice [28] — while the permutation null is computed through the identical pipeline on shuffled labels. Reported as null. Component importances were not examined, because interpreting coefficient loadings under a null result would be fishing.

Interpretation. With 140 clips against 30 to 110 dimensions and a 5.3 mph baseline spread, only a large effect is detectable, so absence of signal here cannot be distinguished from insufficient power. Piergiovanni and Ryoo [12] attempt the same prediction from the same broadcast footage and describe it as extremely difficult, which is consistent. The null is not a claim that pitch velocity lacks kinematic correlates: laboratory motion capture does separate high- from low-velocity pitchers on kinematic and temporal parameters [13]. What this result bears on is whether those correlates survive single-view broadcast pose at this sample size, and here they do not. This endpoint should be revisited if the clip pool grows substantially, and is reported here as a pre-registered negative rather than omitted.

IX. Evaluation lessons

Methodological finding on evaluation. Five intermediate results in this project were false positives. Four were

TABLE XIX
ALL FIVE REPRESENTATIONS AGAINST THE T1 BASELINE OF -0.0035 AND THE T2 BASELINE OF 0.4919, WITH PERMUTATION P-VALUES. NOT ONE CLEARS EITHER BAR. MEAN ABSOLUTE ERROR ON T1 RANGES 5.32 TO 5.37 MPH AGAINST A 5.32 MPH MEAN-PREDICTOR BASELINE.

representation	dims	T1 speed, CV R2	p	T2 class, bal. acc.	p
R0 kinematic summary (reference)	70	-0.0179	0.63	0.4546	0.82
R1 per-clip SINDy coefficients	110	-0.0218	0.63	0.5504	0.19
R2 per-clip weak-form coefficients	110	-0.0035	0.33	0.5000	1.00
R3 DMD spectrum	30	-0.0133	0.54	0.4373	0.88
R4 Hankel (time-delay) DMD spectrum	30	-0.0201	0.64	0.4833	0.67

caught by adding the no-model control for the specific task being scored; the fifth by a null the analysis had not originally applied per target:

A sixth near-miss was an aggregation artifact rather than a missing control: pooled skill is positive at every rollout horizon while uniform skill is negative at every horizon (Section V), and reporting only the pooled figure would have produced another spurious positive. The second-order row above was anticipated before the run — position-increment prediction is trivial when velocity is a state variable — so the control was built in and the artifact was caught on first inspection rather than after it had been written up.

The common structure is that all five involved metrics whose absolute values are uninterpretable without a reference point — R2 against a window mean when the target barely moves, and sum-of-squares pooled across columns of very different amplitude. The practice adopted here, and the one this project would recommend to similar work, is that a metric ships with its no-model control and both aggregations before any model is scored against it, not after a result looks surprising. Two established results underwrite it: a score expressed against a trivial reference forecast is interpretable where an absolute one is not [23], and a permutation null computed through the identical pipeline is the test that survives when the same folds have also informed a modelling choice [27], [28].

A. A selection rule that manufactured a finding

An earlier internal comparison recorded the weak-form model as the best available on the fair metric (core R2 -7.89 versus -29.56) and recommended it as operative. Regenerating that run shows the result holds numerically but does not support that conclusion. The selected weak-form model has a **single non-bias term** across the entire 10-dimensional state:

```
d(dominant_knee_flexion_angle)/dt = 0.354 *
↪  trail_elbow_flexion_angle
```

All nine other state derivatives are identically zero. The model is, in substance, the frozen control with one residual term — and it scores *worse* than the true frozen control.

The cause is a degeneracy in the model-selection rule. From the weak-form selection log (`results/equations/baseball/train_weakform_2026-08-12/pitching/model_selection.csv`):

Every candidate's validation R2 lies in a band of width 0.0013 around zero, so the accuracy term carries no signal. Selection is therefore decided entirely by the sparsity penalty, and threshold 0.30 wins over 0.20 by **1.5e-05**. When all candidates are equally uninformative, `sparsity_weighted_r2` reduces to "prefer the emptiest model." The apparent weak-form victory is an artifact of that rule interacting with a metric that rewards not moving, not evidence that the weak-form fit recovered better dynamics.

This is why the frozen control is necessary. Without it, a near-null model reads as a 3.7x improvement.

The rule is now repaired. `pick_best_result` gains an `accuracy_gated_sparsity` metric that keeps every candidate within a configurable tolerance of the best accuracy and takes the sparsest of those, so sparsity breaks ties rather than overruling accuracy. Independently, the fitter now computes a `selection_degenerate` flag: if no candidate reaches `selection_min_accuracy` (default 0, since a candidate below zero is worse than predicting the mean), the sweep had nothing to discriminate on and the choice should not be interpreted. The flag is written to `summary.csv` and `config.json` and printed as a warning. Both changes are additive, so the Stage A fitter's behaviour is unchanged unless a config opts in.

Two things follow. The repaired rule would not have produced the near-null weak-form model: it selects the most accurate candidate rather than the emptiest. And **the operative first-order configuration selects identically under both rules**, so the paper's headline model was never distorted by the defect — only the weak-form comparison was. Both runs are nonetheless flagged degenerate, which is correct: no candidate on either sweep reaches an accuracy above zero, so neither selection means anything beyond "these candidates are indistinguishable".

Selecting on skill over persistence does not rescue

TABLE XX
Intermediate results that did not survive the control or null for their task.

apparent result	control that overturned it	outcome
weak-form core R2 -7.89, "3.7x better than baseline"	frozen zero-dynamics model, -5.57	dissolved; the model had one non-bias term
delay-embedding derivative R2 +0.69	unfitted backward difference, +0.35	roughly half was numerical differentiation
teacher-forced one-step core R2 +0.76	persistence, +0.85	dissolved
second-order short-horizon skill +0.89	its own derivative reference, +0.89	dissolved; velocity is an input column
proximal-to-distal asymmetry +0.0011, four of five splits clearing a shift null	the same null applied per target, over 30 resplits	dissolved; interval [-0.0004, +0.0026], positive on 18 of 30

TABLE XXI
Weak-form model selection log. All candidates are indistinguishable in validation R2, so the sparsity penalty decides and threshold 0.30 wins by 1.5e-05.

threshold	terms	validation mean trial R2	selection score
0.10	37	-0.008702	-0.010552
0.15	23	-0.008969	-0.010119
0.20	13	-0.009013	-0.009663
0.30	**2**	**-0.009548**	**-0.009648**
0.50	0	-0.010045	-0.010045

TABLE XXII
Selection under the old and repaired rules. The weak-form choice changes completely; the operative configuration does not.

configuration	`sparsity_weighted_r2`	`accuracy_gated_sparsity`	flagged degenerate
weak-form	threshold 0.30, **1 term**	threshold 0.10, **29 terms**	yes
first-order (operative)	threshold 0.20, 26 terms	threshold 0.20, 26 terms	yes

the sweep, and the reason is instructive. The fitter can now select on `persistence_skill_uniform`, computed on the validation clips during the sweep. The expectation was that it would discriminate where derivative R2 cannot. On the first-order state it does not: across the five thresholds the skill spans just 0.00088, from -0.00383 to -0.00471, every value negative. Selection remains degenerate and is correctly flagged.

That is not a failure of the criterion. On the delay-augmented state of Section VII the same metric spans **0.0378** across the same thresholds (+0.3881 down to +0.3502), forty times the spread, and picks a clear winner. A selection criterion can only separate candidates that actually differ, and first-order candidates do not differ because none of them predicts anything — which is Section VI's conclusion arriving from a third direction. The rule was worth repairing, but no rule can make a sweep meaningful when every candidate is equally useless.

B. When two aggregations disagree

Pooled skill is positive at every horizon; uniform skill is negative at every horizon, and uniform core R2 favours persistence at every horizon. Per Section IV this is reported as **no multi-step win**. The disagreement is fully explained by the amplitude spread across core columns: pooled sum-of-squares is decided by the large-amplitude angles, on which the model does help, while the small-amplitude columns on which it does badly count equally under uniform averaging.

The H=240 row is the only sign of the crossover one would expect as persistence degrades — core R2 -1.87 against persistence -2.73 — but uniform skill is still negative there and only 13 clips are long enough to qualify. It is recorded as a direction to test with longer clips, not as a result.

X. Limitations and future work

- The dataset is 140 clips from a single camera view, and may not cover enough athlete, view, and motion variability to support a global model.
- MediaPipe-derived pose from broadcast or segmented video is noisier than marker-based motion capture.
- Many clips are effectively 2D pose observations, which limits interpretation of out-of-plane motion.
- Camera angle, zoom, and occlusion may change feature values independently of true biomechanics.
- The polynomial libraries tried are low-degree; the second-order variants that add expressivity performed far worse on held-out reconstruction, so underfitting is not the binding constraint on the current data.
- No configuration has demonstrated predictive validity on free-running reconstruction. The Section VII result establishes short-horizon predictive structure only, and does not license claims about longer horizons.
- The Section VII skill figure comes from a delay-embedded ridge model, not from a sparse SINDy fit. Section VI tests the fitted equations directly and finds they capture 0 percent (first-order) to 12 percent (second-order, acceleration) of the structure available in their own training targets. The short-horizon result therefore characterizes the data, not the pipeline's sparse artifacts, and must not be attributed to SINDy.
- Skill over persistence measures prediction of increments. It is not a biomechanical validity measure,

and 0.38 skill does not imply the fitted coefficients correspond to real biomechanical relationships.
- The H=240 horizon result rests on the 13 clips long enough to qualify, and is reported as a direction rather than a finding.
- The noise-floor estimate is computed on already-smoothed features, so it bounds achievable precision in the modeled state rather than measuring raw MediaPipe jitter.
- The Section VII result is a one-step increment model on a delay-augmented state. It is not a free-running dynamical system and cannot be integrated forward; Section V shows what happens when such a model is rolled out. It also rests on a single train/test split with no confidence interval.
- The cross-joint coupling gain is statistically detectable but is about 1.8 percent of the delay model's skill, and is absent on one held-out split in five. It shows that joints are coupled, which is expected of any linked body. The directional asymmetry that would evidence the proximal-to-distal chain was tested and does not survive: +0.0011, CI [-0.0004, +0.0026], positive on 18 of 30 splits.
- The chain ordering used in that test follows the throwing sequence described by Fleisig et al. [2] — stride, pelvis, upper torso, elbow, shoulder, wrist — rather than the data. A different defensible ordering could change the result.
- The Section VIII null cannot distinguish absence of label signal from insufficient power at 140 clips.
- The `sparsity_weighted_r2` selection rule degenerates to a term-count minimizer when candidate accuracies are indistinguishable (Section IX); selected sparsity levels should not be read as evidence of a parsimonious true model.
- Sections V and VI-C report the operative pipeline's single train/test split, since that is what the pipeline produces. Sections V, 6.8 and 6.9 now carry intervals — paired bootstrap over clips for the control comparison, resplits elsewhere — but the ensemble and second-order rows of Section V have not been re-scored with intervals, because that would mean refitting those configurations on every resample.
- The system identifies sparse equations from measured features; it does not prove causal biomechanical mechanisms.

Ordered by what would most change the paper's conclusions.

1. **Decide whether the delay-augmented configuration becomes the operative default.** It is now a first-class pipeline configuration (Section VII) and beats the current default on every metric that discriminates: derivative R2 +0.4411 against -0.0184, skill +0.4071 against -0.0041, and a non-degenerate candidate sweep. What holds the decision open is that it answers a different question — one-step prediction rather than free-running reconstruction — so switching means changing what the pipeline is for. That is an editorial call, not a technical one.
2. **Score the ensemble and second-order configurations against the frozen control with paired intervals.** Section V's two remaining rows are still single-split, because unlike the others they need refitting per resample.
3. **Extend the horizon sweep with longer clips** to test whether the crossover hinted at H = 240 (Section V, 13 clips) is real. It is the one place a fitted model came close to the control.
4. **Re-score the alignment track** under the current pool and the core-position metric (Section A). It has never been evaluated on comparable footing, so the unaligned default rests on a superseded comparison.
5. **Add automated pose-quality metrics** and exclude low-confidence or heavily occluded clips, which is the one untested route to raising the ceiling on the pitching data.
6. **Batting needs a different pose model, and this is now measured rather than suspected.** The batter is detected at all in only 78 percent of clips, and the best of seven selection rules reaches 71 percent on the clips where he is, so roughly half of a batting dataset would be the umpire or the pitcher. Upscaling the plate crop to 3x and 4x and loosening the detection threshold do not fix it: on the clips where the batter is missed he remains missed, while the umpire and catcher beside him are found, even though he is large, unoccluded and well lit at that scale. The limit is therefore the detector on this subject and not the input resolution, so the next step is a different pose model or an upstream person detector feeding single-person crops — the top-down arrangement that high-resolution pose networks are built around [29] — not another selection heuristic. Cropping the plate region, truncating at camera-pan onset, and the labelled set for scoring rules are all worth keeping.

Standing practice rather than tasks: report the no-model control for the specific task being scored, under both aggregations, and treat sign disagreement between them as a null result.

Explicitly deprioritized on evidence gathered here: per-pitcher, per-pitch-type and per-phase conditioning (the per-clip oracle in Section VI shows heterogeneity is not the constraint); non-autonomous or phase-augmented state vectors (no gain in three forms); further sparse-estimator substitution (dense and sparse intervals overlap, Section VI); and further representations for the label study: all five of the pre-registered set have now been evaluated and none clears either bar (Section VIII), including the delay-embedded spectrum that Section VI's finding would predict to be the most promising.

XI. Conclusion

Free-running reconstruction of broadcast-derived pitching pose is not achievable with sparse dynamical models: every configuration tried is beaten by a predictor that assumes the pitcher does not move, and accuracy falls as model complexity rises. That failure is localized rather than merely reported. Phase, clip heterogeneity, the pose noise floor and the choice of sparse estimator were each excluded by direct test, leaving the state representation — and supplying two frames of history raises skill over persistence on the identical target from +0.01 to +0.41, which the pipeline now reproduces end to end from a configuration file.

What the data does not support is equally definite. Release velocity and pitch type are not predictable from any of the five pre-registered representations, and the directional asymmetry that would evidence a proximal-to-distal kinetic chain does not survive its null. This paper therefore makes no biomechanical claim, and the discovered coefficients should not be read as mechanics.

The evaluation protocol is the transferable part. Five intermediate results in this project were false positives: four were caught by adding the no-model control for the specific task being scored, the fifth by a null the analysis had not originally applied per target, and a sixth near-miss was an aggregation artifact. A metric whose absolute value is uninterpretable — R2 against a window mean when the target barely moves, or a sum of squares pooled across columns of unequal amplitude — will manufacture findings unless it ships with that control and both aggregations before any model is scored against it. On this evidence that discipline is not optional bookkeeping; it is the difference between a negative result and a wrong one.

The defensible claim is:

> A reproducible sparse-dynamics pipeline for baseball pitching pose data was implemented end-to-end, together with a state-comparable reconstruction metric and a control-paired evaluation protocol in which every reported score is compared against the no-model baseline for its own task. Under that protocol three findings separate. First, no fitted configuration — first-order, second-order, ensemble, or weak-form — recovers dynamics that survive free-running reconstruction; all are outperformed by a zero-dynamics baseline, and accuracy degrades monotonically with model complexity. Second, the data does carry short-horizon structure: a delay-embedded linear model achieves 0.39 skill over persistence on held-out increment prediction, robust to error aggregation and backed by increment autocorrelation of 0.35 to 0.88 rather than by pose noise. Third, the pipeline's own sparse equations capture almost none of that structure as configured, but the cause is the state and not the estimator: the same sparse estimator on a delay-augmented state reaches +0.3881 uniform skill over persistence with 30 terms, matching dense ridge and clearing an unfitted momentum control at +0.1311. That state is now a first-class pipeline configuration, and running the standard commands on it yields +0.4071 skill and the project's first positive derivative R2 (+0.4411).

Two things this work must not claim. The results are not evidence of recovered governing laws of pitching mechanics: 26 coefficients that predict increments 39 percent better than persistence are not a biomechanical model, and the discovered equations have not been validated against any external quantity. And the weak-form configuration must not be presented as a successful model.

This work contributes a reproducible software framework for sparse dynamical modeling of baseball pitching motion from standardized pose trajectories, together with an evaluation protocol for it. The engineering contribution is an end-to-end pipeline that transforms vision-derived skeletal motion into interpretable sparse-equation artifacts with documented preprocessing, feature extraction, model fitting, and evaluation, reproducible from scripted commands in the sense set out for computational research [30]: every number in Section V regenerates from a command in Section XII, and generated artifacts are excluded from version control rather than committed alongside them.

The methodological contribution is the control-paired evaluation protocol: a state-comparable core-position reconstruction metric, a frozen-model control and a persistence control that make it interpretable, dual aggregation with sign disagreement treated as a null, and the discipline of matching each control to the specific task being scored. This protocol changed four conclusions in this project — three apparent positive results proved to be artifacts, and one real result had been invisible under the metric previously used (Sections IX, 6.4, 6.5, 6.6). That track record is the strongest argument for the protocol.

The empirical contribution is a characterized boundary rather than a flat negative, with the failure localized. Across first-order, second-order, ensemble, and weak-form configurations on 140 single-view broadcast-derived pitching clips, no fitted sparse model survives free-running reconstruction, and all are outperformed by a zero-dynamics baseline. Yet the same data supports short-horizon prediction: 0.39 skill over persistence on held-out increments, backed by increment autocorrelation of 0.35 to 0.88 and signal-to-noise of 2.2 to 4.4. Candidate explanations were tested and excluded one at a time — phase (no gain in three forms), clip heterogeneity (the per-clip oracle scores below a cross-clip delay model), the pose noise floor (SNR 2.2 to 4.4), and the sparse estimator (dense ridge and the best STLSQ agree to within 0.0003, Section VI). What remains is the state representation: an autonomous first-order state of joint positions does not determine its own next increment, and supplying two frames of history raises skill on the identical target from +0.0099 to +0.3874. That

is a specific, testable diagnosis that points at a concrete change rather than at more tuning.

The practical value of the boundary is that it says what to build next, and what not to. Short-horizon prediction is a task this data supports and is where a defensible biomechanical claim would have to start, but the sparse-fitting stage must be repaired before the pipeline can make that claim on its own artifacts. Free-running reconstruction of broadcast-derived pitching pose is not achievable with these methods and should not be the target. Better pose, larger pools, richer libraries, and per-pitcher conditioning are all secondary to the sparse fit, since none of them is where the measured loss occurs.

XII. Reproducibility

All commands are run from the project root with the repository virtual environment. **The operative configuration must be passed explicitly** — the refresh script's default (`config/preprocessing.baseball.example.yaml`) does not reproduce Section V.

Install dependencies:

```
./.venv/bin/python -m pip install -r requirements.txt
```

Validate Python syntax:

```
./.venv/bin/python -m compileall src
```

Preview the operative baseball pipeline:

```
./.venv/bin/python src/preprocessing/refresh_baseball_pipeline.py \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
  ↪  --dry-run
```

Reproduce Section V (operative first-order train + held-out evaluation):

```
./.venv/bin/python src/preprocessing/refresh_baseball_pipeline.py \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
```

Reproduce Section IX (weak-form model). Note that fit and evaluation write to the same `results/equations/baseball/train/` and `results/evaluation/baseball/train_on_test/` paths for every configuration, so snapshot or re-run the baseline afterwards if both are needed:

```
./.venv/bin/python src/preprocessing/fit_baseball_sindy.py \
  --config config/preprocessing.baseball.unaligned.handedness.weak⌋
  ↪  form.yaml
  ↪  \
  --split train --task-types pitching
./.venv/bin/python src/preprocessing/evaluate_baseball_sindy.py \
  --config config/preprocessing.baseball.unaligned.handedness.weak⌋
  ↪  form.yaml
  ↪  \
  --train-split train --eval-split test --task-types pitching
```

Reproduce the frozen control in Section V:

```
./.venv/bin/python src/preprocessing/evaluate_static_baseline.py \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
  ↪  --eval-split test
```

Reproduce the state-representation sweep in Section VI:

```
./.venv/bin/python src/preprocessing/diagnose_derivative_fit.py \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
```

Reproduce Sections VII, 6.6 and 6.7 (one-step skill, horizon sweep, noise floor):

```
./.venv/bin/python src/preprocessing/diagnose_increment_skill.py \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
```

Reproduce the free-running control comparison in Section V:

```
./.venv/bin/python src/preprocessing/evaluate_forecast_baselines.py
↪  \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
```

Reproduce Section VI (fitted SINDy equations on the short-horizon task):

```
./.venv/bin/python
↪  src/preprocessing/evaluate_sindy_increment_skill.py \
  --config config/preprocessing.baseball.unaligned.handedness.yaml
  ↪  \
  --model-root results/equations/baseball/train/pitching
```

For the second-order state, extract to an isolated feature subdir first so the 10-D features every other result depends on are not overwritten, then score the non-trivial task:

```
for split in train test; do
  ./.venv/bin/python src/preprocessing/extract_baseball_features.py
  ↪  \
    --config config/preprocessing.baseball.unaligned.second_order.⌋
    ↪  isolated.yaml --split
    ↪  $split
done
./.venv/bin/python
↪  src/preprocessing/evaluate_sindy_increment_skill.py \
  --config config/preprocessing.baseball.unaligned.second_order.is⌋
  ↪  olated.yaml
  ↪  \
  --model-root results/equations/baseball/train_second_order_2026-⌋
  ↪  06-23/pitching
  ↪  \
  --column-set rates
```

Reproduce Section VII (sparse fit on a delay-augmented state, with the momentum controls and the coupling ablation):

```
./.venv/bin/python src/analysis/fit_delay_sindy.py
```

Reproduce Section VIII (label-validated representation study). This runs 200 permutations per representation per task and takes several minutes; `--quick` reduces it for a smoke run:

```
./.venv/bin/python src/analysis/run_representation_study.py
```

Reproduce the Section VI estimator comparison:

```
./.venv/bin/python src/analysis/diagnose_sparse_fit_loss.py
```

Reproduce the Section VII coupling test (five held-out splits, 60 null draws each):

```
./.venv/bin/python src/analysis/test_coupling_significance.py
↪  --n-null 60 --null-splits 5
```

Regenerate the figures. They are build artifacts under `results/`, which is excluded from version control, so this must run before the paper is built:

```
./.venv/bin/python src/analysis/make_paper_figures.py
```

Run the delay-augmented configuration end to end (Section VII):

```
for split in train test; do
  ./.venv/bin/python src/preprocessing/extract_baseball_features.py
  ↪  \
    --config config/preprocessing.baseball.delay.yaml --split
    ↪  $split
done
./.venv/bin/python src/preprocessing/fit_baseball_sindy.py \
```

```
  --config config/preprocessing.baseball.delay.yaml --split train
  ↪  --task-types pitching
./.venv/bin/python src/preprocessing/evaluate_baseball_sindy.py \
  --config config/preprocessing.baseball.delay.yaml \
  --train-split train --eval-split test --task-types pitching
```

Compare the old and repaired selection rules (Section IX):

```
./.venv/bin/python src/preprocessing/fit_baseball_sindy.py \
  --config config/preprocessing.baseball.unaligned.handedness.weak⌋
  ↪  form.yaml
  ↪  \
  --split train --task-types pitching
./.venv/bin/python src/preprocessing/fit_baseball_sindy.py \
  --config config/preprocessing.baseball.unaligned.handedness.weak⌋
  ↪  form.gated.yaml
  ↪  \
  --split train --task-types pitching
```

Reproduce the Section V paired control intervals (requires the per-clip control scores, which `evaluate_static_baseline.py` writes alongside its summary):

```
./.venv/bin/python src/preprocessing/evaluate_static_baseline.py
./.venv/bin/python src/analysis/paired_control_intervals.py
```

Reproduce the Section VII stability comparison:

```
./.venv/bin/python src/analysis/stabilize_delay_fit.py --n-splits
↪  40
```

Reproduce the Section VII directional asymmetry test:

```
./.venv/bin/python src/analysis/test_kinetic_chain_asymmetry.py
↪  --n-null 60 --null-splits 5
```

Compare multiple configurations in one run:

```
./.venv/bin/python src/preprocessing/compare_baseball_configs.py \
  --configs config/preprocessing.baseball.unaligned.handedness.yaml
  ↪  \
           config/preprocessing.baseball.unaligned.handedness.wea⌋
           ↪  kform.yaml
           ↪  \
  --output-csv
  ↪  results/evaluation/baseball/config_comparison_weakform.csv
```

Primary generated outputs:

- `data/baseball/processed/train/features/manifest.csv`
- `data/baseball/processed/test/features/manifest.csv`
- `results/equations/baseball/train/summary.csv`
- `results/equations/baseball/train/pitching/coefficients.csv`
- `results/equations/baseball/train/pitching/equations.txt`
- `results/equations/baseball/train/pitching/model_selection.csv`
- `results/evaluation/baseball/train_on_test/summary.csv`
- `results/evaluation/baseball/train_on_test/reconstruction_metrics.csv`
- `results/evaluation/baseball/static_baseline/summary.csv`
- `results/evaluation/baseball/model_comparison_2026-08-12.csv`
- `results/evaluation/baseball/forecast_baselines/summary.csv`
- `results/analysis/derivative_fit_diagnostics.csv`
- `results/analysis/increment_skill/part_a_parameterization.csv`
- `results/analysis/increment_skill/part_b_horizon_crossover.csv`
- `results/analysis/increment_skill/part_c_noise_floor.csv`
- `results/analysis/sindy_increment_skill.csv`
- `results/analysis/sindy_increment_skill_second_order_rates.csv`
- `results/analysis/sparse_fit_loss.csv`
- `results/analysis/delay_sindy/skill_by_depth.csv`
- `results/analysis/delay_sindy/equations.txt`
- `results/analysis/coupling_significance/summary.json`
- `results/analysis/delay_sindy/stability.csv`
- `results/analysis/kinetic_chain_asymmetry/summary.json`
- `results/analysis/paired_control_intervals.csv`
- `results/evaluation/baseball/static_baseline/per_clip.csv`
- `results/analysis/representation_study/comparison.csv`

Analysis code is Python 3.12. Numerical work uses NumPy [31] and SciPy [32]; Savitzky-Golay smoothing and differentiation use `scipy.signal.savgol_filter`. Ridge regression, cross-validation, standardization, and R2 computation use scikit-learn [33]. Sparse model identification uses PySINDy 2.1.0 [9], [10]. Pose extraction uses MediaPipe [19]. Exact pinned versions are in `requirements.txt`.

XIII. A. Appendix: superseded and auxiliary results

Retained for provenance. Nothing here supports a claim in the body.

A. A.1 Preprocessing-track comparison

An earlier aligned-versus-unaligned comparison (`results/evaluation/baseball/config_comparison.csv`) reported the unaligned track at 78 terms / whole-state R2 -4.8905 and the aligned track at 98 terms / whole-state R2 -2.4099. That comparison predates both the restored 140-clip pool and the core-position metric, and it ranked tracks on the whole-state score that Section IV shows is not comparable across configurations. It is retained for history and is **not** the basis of the current operative choice. The alignment track has not been re-scored under the current pool and metric; doing so is listed in Section X.

B. A.2 Delay-model stability

Instability, diagnosed and fixed. The unrestricted 30-column fit diverged on 2 of 40 random clip-level splits, producing predictions of order 1e4 against true increments of order 1. On a diverging split the design has

condition number 3.0e6 (smallest singular value 8.6e-5) and the fitted coefficients reach ±6773 on exactly three columns — `d1(shoulder_rotation_angle)`, `d1(hip_rotation_angle)` and `d1(hip_shoulder_separation_angle)` — which cancel. On a healthy split the same design has condition number 6.1 and coefficients of order 0.4.

The cause is that `hip_shoulder_separation_angle` is *derived*: it is shoulder rotation minus hip rotation. Pooled over all 140 clips the identity is not exact (residual standard deviation 1.03 against the feature's own 1.11, and the ten-feature matrix has condition number 51 at full rank), which is why it is invisible in aggregate. It holds near-exactly on *some clip subsets*, so a training draw that happens to contain mostly such clips is rank-deficient in those three columns, and least squares answers with enormous cancelling coefficients that do not transfer.

`src/analysis/stabilize_delay_fit.py` compares candidate fixes over the same 40 splits (`results/analysis/delay_sindy/stability.csv`):

Raising STLSQ's ridge parameter does not help even at alpha = 1e3, which rules out ordinary ill-conditioning as the mechanism and confirms a genuine rank deficiency. Whitening fixes it but replaces 32 physical terms with 111 principal components and costs 0.009 skill. **Dropping the derived feature is the operative fix**: divergence goes to zero at a cost of 0.0009 skill, well inside the confidence interval, and the model stays interpretable. Nothing is lost by removing it, since the two rotations that define it remain in the design and it is not a target column.

The delay model is therefore stable at 27 design dimensions and about 32 terms. The coupling analysis in Section VIII-A predates this fix, which is why it had to exclude diverging splits by hand; repeating it on the stabilized design is listed in Section X.

XIV. References

[1] G. S. Fleisig, J. R. Andrews, C. J. Dillman, and R. F. Escamilla, "Kinetics of baseball pitching with implications about injury mechanisms," *Amer. J. Sports Med.*, vol. 23, no. 2, pp. 233–239, Mar./Apr. 1995, doi: 10.1177/036354659502300218.

[2] G. S. Fleisig, S. W. Barrentine, R. F. Escamilla, and J. R. Andrews, "Biomechanics of overhand throwing with implications for injuries," *Sports Med.*, vol. 21, no. 6, pp. 421–437, Jun. 1996, doi: 10.2165/00007256-199621060-00004.

[3] R. M. Kanko, E. K. Laende, E. M. Davis, W. S. Selbie, and K. J. Deluzio, "Concurrent assessment of gait kinematics using marker-based and markerless motion capture," *J. Biomech.*, vol. 127, art. no. 110665, Oct. 2021, doi: 10.1016/j.jbiomech.2021.110665.

[4] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci.*, vol. 113, no. 15, pp. 3932–3937, Apr. 2016, doi: 10.1073/pnas.1517384113.

[5] D. A. Messenger and D. M. Bortz, "Weak SINDy: Galerkin-based data-driven model selection," *Multiscale Model. Simul.*, vol. 19, no. 3, pp. 1474–1497, 2021, doi: 10.1137/20M1343166.

[6] D. A. Messenger and D. M. Bortz, "Weak SINDy for partial differential equations," *J. Comput. Phys.*, vol. 443, art. no. 110525, 2021, doi: 10.1016/j.jcp.2021.110525.

[7] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, "Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control," *Proc. Roy. Soc. A*, vol. 478, no. 2260, art. no. 20210904, 2022, doi: 10.1098/rspa.2021.0904.

[8] K. Champion, P. Zheng, A. Y. Aravkin, S. L. Brunton, and J. N. Kutz, "A unified sparse optimization framework to learn parsimonious physics-informed models from data," *IEEE Access*, vol. 8, pp. 169259–169271, 2020, doi: 10.1109/ACCESS.2020.3023625.

[9] B. M. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. N. Kutz, and S. L. Brunton, "PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data," *J. Open Source Softw.*, vol. 5, no. 49, art. no. 2104, 2020, doi: 10.21105/joss.02104.

[10] A. A. Kaptanoglu et al., "PySINDy: A comprehensive Python package for robust sparse system identification," *J. Open Source Softw.*, vol. 7, no. 69, art. no. 3994, 2022, doi: 10.21105/joss.03994.

[11] M. C. Rosenberg, J. L. Proctor, and K. M. Steele, "Quantifying changes in individual-specific template-based representations of center-of-mass dynamics during walking with ankle exoskeletons using Hybrid-SINDy," *Sci. Rep.*, vol. 14, art. no. 1031, 2024, doi: 10.1038/s41598-023-50999-0.

[12] A. J. Piergiovanni and M. S. Ryoo, "Fine-grained activity recognition in baseball videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR) Workshops*, 2018. [Online]. Available: https://arxiv.org/abs/1804.03247

[13] T. Matsuo, G. S. Fleisig, R. F. Escamilla, S. W. Barrentine, and J. R. Andrews, "Comparison of kinematic and temporal parameters between different pitch velocity groups," *J. Appl. Biomech.*, vol. 17, no. 1, pp. 1–13, 2001.

[14] A. L. Aguinaldo, J. Buttermore, and H. Chambers, "Effects of upper trunk rotation on shoulder joint torque among baseball pitchers of various levels," *J. Appl. Biomech.*, vol. 23, no. 1, pp. 42–51, Feb. 2007, doi: 10.1123/jab.23.1.42.

[15] F. Takens, "Detecting strange attractors in turbulence," in *Dynamical Systems and Turbulence, Warwick 1980* (Lecture Notes in Mathematics, vol. 898), D. A. Rand and L.-S. Young, Eds. Berlin, Germany: Springer, 1981, pp. 366–381, doi: 10.1007/BFb0091924.

TABLE XXIII
CANDIDATE FIXES FOR THE DIVERGENCE, EVALUATED OVER IDENTICAL SPLITS. ONLY REMOVING THE DERIVED FEATURE OR WHITENING THE DESIGN ELIMINATES IT; REMOVING THE DERIVED FEATURE COSTS NOTHING AND KEEPS THE TERMS INTERPRETABLE.

variant	dims	diverged	skill	95% CI	terms
baseline	30	2/40	+0.3813	[+0.3676, +0.3951]	33.6
STLSQ ridge alpha 1e-1 / 1.0 / 1e3	30	2/40	+0.3820	[+0.3682, +0.3957]	32.8
levels centred per clip	30	2/40	+0.3852	[+0.3716, +0.3988]	34.6
increments only	20	2/40	+0.3678	[+0.3536, +0.3819]	24.2
PCA-whitened design	30	**0/40**	+0.3720	[+0.3593, +0.3846]	110.7
drop `hip_shoulder_separation_angle`	27	**0/40**	**+0.3805**	[+0.3670, +0.3940]	32.1

[16] S. L. Brunton, B. W. Brunton, J. L. Proctor, E. Kaiser, and J. N. Kutz, "Chaos as an intermittently forced linear system," *Nature Commun.*, vol. 8, art. no. 19, 2017, doi: 10.1038/s41467-017-00030-8.

[17] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *J. Fluid Mech.*, vol. 656, pp. 5–28, Jul. 2010, doi: 10.1017/S0022112010001217.

[18] C. Chen, R. Jafari, and N. Kehtarnavaz, "UTD-MHAD: A multimodal dataset for human action recognition utilizing a depth camera and a wearable inertial sensor," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2015, pp. 168–172, doi: 10.1109/ICIP.2015.7350781.

[19] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019. [Online]. Available: https://arxiv.org/abs/1906.08172

[20] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020. [Online]. Available: https://arxiv.org/abs/2006.10204

[21] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Anal. Chem.*, vol. 36, no. 8, pp. 1627–1639, 1964, doi: 10.1021/ac60214a047.

[22] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679–688, 2006, doi: 10.1016/j.ijforecast.2006.03.001.

[23] A. H. Murphy, "Skill scores based on the mean square error and their relationships to the correlation coefficient," *Mon. Weather Rev.*, vol. 116, no. 12, pp. 2417–2424, Dec. 1988, doi: 10.1175/1520-0493(1988)116<2417:SSBOTM>2.0.CO;2.

[24] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[25] S. Ross and J. A. Bagnell, "Efficient reductions for imitation learning," in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, PMLR, 2010, pp. 661–668.

[26] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Inf. Sci.*, vol. 191, pp. 192–213, 2012, doi: 10.1016/j.ins.2011.12.028.

[27] M. Ojala and G. C. Garriga, "Permutation tests for studying classifier performance," *J. Mach. Learn. Res.*, vol. 11, pp. 1833–1863, 2010.

[28] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, art. no. 91, 2006, doi: 10.1186/1471-2105-7-91.

[29] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for human pose estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 5693–5703.

[30] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research," *PLoS Comput. Biol.*, vol. 9, no. 10, art. no. e1003285, 2013, doi: 10.1371/journal.pcbi.1003285.

[31] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[32] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[33] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.